

Hoja de Trabajo - Laboratorio No. 2

Nombre: Luna Samantha Arrazola Escobar

Carné: 1022725

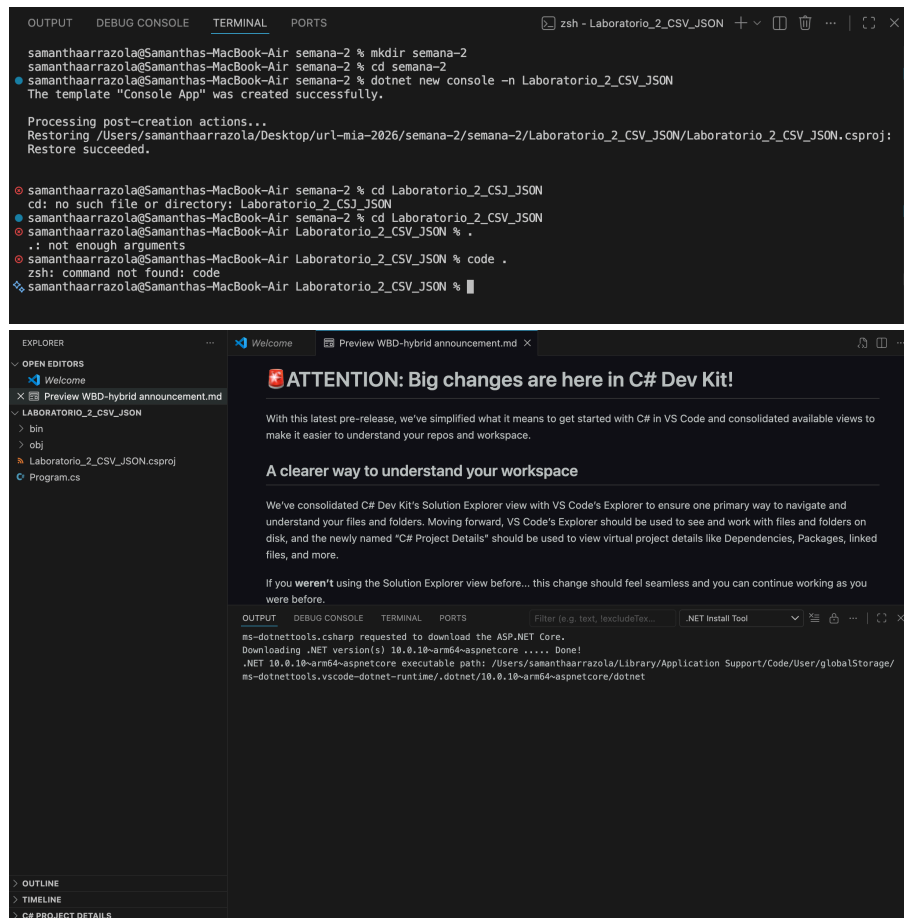
Fecha: 17-07-2026

Práctica: Formatos de Archivos

Objetivo: Identificar tipos de archivos mas comunes en programación.

1. CREACIÓN DEL PROYECTO

- Utilizar Visual Studio code.
- Cree un proyecto tipo Aplicación de Consola (.NET).
- Asigne el nombre: Laboratorio_2_CSV_JSON



The screenshot displays the Visual Studio Code interface. The top panel shows the TERMINAL view with the following output:

```
samanthaarrazola@Samanthas-MacBook-Air semana-2 % mkdir semana-2
samanthaarrazola@Samanthas-MacBook-Air semana-2 % cd semana-2
samanthaarrazola@Samanthas-MacBook-Air semana-2 % dotnet new console -n Laboratorio_2_CSV_JSON
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring /Users/samanthaarrazola/Desktop/url-mia-2026/semana-2/semana-2/Laboratorio_2_CSV_JSON/Laboratorio_2_CSV_JSON.csproj:
Restore succeeded.

samanthaarrazola@Samanthas-MacBook-Air semana-2 % cd Laboratorio_2_CSV_JSON
cd: no such file or directory: Laboratorio_2_CSV_JSON
samanthaarrazola@Samanthas-MacBook-Air semana-2 % cd Laboratorio_2_CSV_JSON
samanthaarrazola@Samanthas-MacBook-Air Laboratorio_2_CSV_JSON % .
.: not enough arguments
samanthaarrazola@Samanthas-MacBook-Air Laboratorio_2_CSV_JSON % code .
zsh: command not found: code
samanthaarrazola@Samanthas-MacBook-Air Laboratorio_2_CSV_JSON %
```

The bottom panel shows the EXPLORER view with the following structure:

- LABORATORIO_2_CSV_JSON
 - bin
 - obj
 - Laboratorio_2_CSV_JSON.csproj
 - Program.cs

The main editor area displays a message about C# Dev Kit changes:

ATTENTION: Big changes are here in C# Dev Kit!

With this latest pre-release, we've simplified what it means to get started with C# in VS Code and consolidated available views to make it easier to understand your repos and workspace.

A clearer way to understand your workspace

We've consolidated C# Dev Kit's Solution Explorer view with VS Code's Explorer to ensure one primary way to navigate and understand your files and folders. Moving forward, VS Code's Explorer should be used to see and work with files and folders on disk, and the newly named "C# Project Details" should be used to view virtual project details like Dependencies, Packages, linked files, and more.

If you weren't using the Solution Explorer view before... this change should feel seamless and you can continue working as you were before.

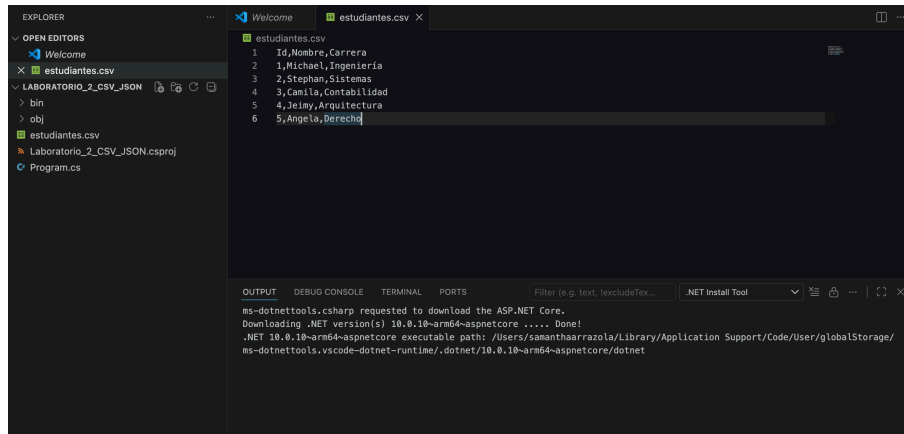
The bottom status bar shows the .NET Install Tool and the output of the command: `ms-dotnettools.csharp requested to download the ASP.NET Core. Downloading .NET version(s) 10.0.10-arm64-aspnetcore Done! .NET 10.0.10-arm64-aspnetcore executable path: /Users/samanthaarrazola/Library/Application Support/Code/User/globalStorage/ms-dotnettools.vsc-dotnet-runtime/.dotnet/10.0.10-arm64-aspnetcore/dotnet`

2. CREACIÓN DEL ARCHIVO CSV

Dentro de la carpeta del proyecto cree un archivo llamado: *estudiantes.csv*

Con el siguiente contenido:

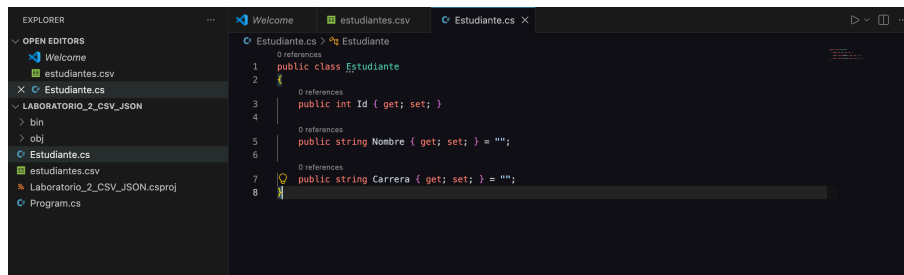
Id,Nombre,Carrera 1,Ana,Ingeniería 2,Carlos,Sistemas 3,María,Contabilidad 4,Pedro,Arquitectura 5,Sofía,Derecho



3. CREACIÓN DE LA CLASE

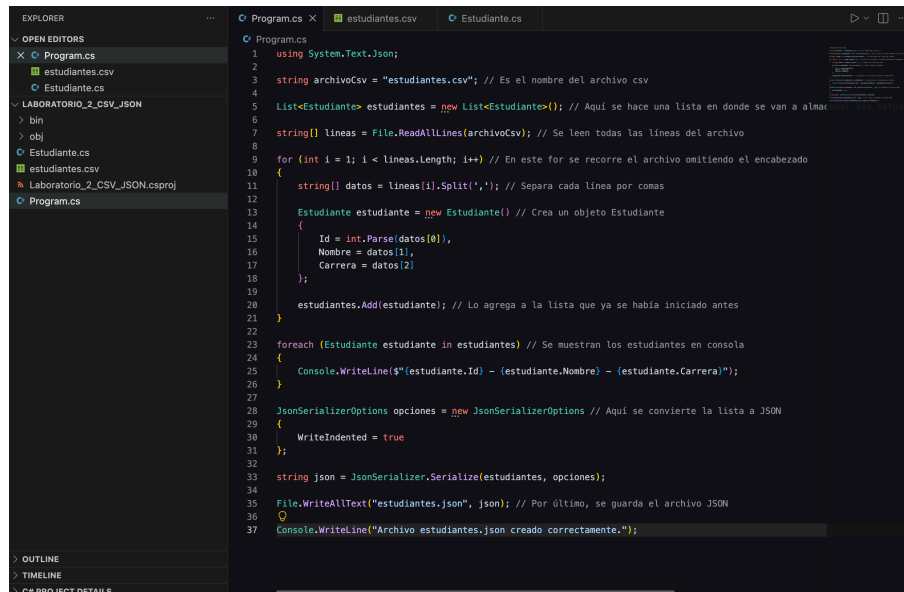
Agregue una clase llamada **Estudiante.cs** con el siguiente código:

```
public class Estudiante { public int Id { get; set; } public string Nombre { get; set; } public string Carrera { get; set; } }
```



4. DESARROLLO DEL PROGRAMA

- Leer el archivo **estudiantes.csv**.
- Omitir la primera línea (encabezado).
- Separar cada registro utilizando el método `Split(',')`.
- Crear objetos de tipo `Estudiante`.
- Guardarlos en una `List<Estudiante>`.
- Mostrar todos los estudiantes en consola.
- Convertir la lista a formato JSON.
- Guardar el resultado en un archivo llamado: `estudiantes.json`



```
1 using System.Text.Json;
2
3 string archivoCsv = "estudiantes.csv"; // Es el nombre del archivo csv
4
5 List<Estudiante> estudiantes = new List<Estudiante>(); // Aquí se hace una lista en donde se van a almacenar los estudiantes
6
7 string[] lineas = File.ReadAllLines(archivoCsv); // Se leen todas las líneas del archivo
8
9 for (int i = 1; i < lineas.Length; i++) // En este for se recorre el archivo omitiendo el encabezado
10 {
11     string[] datos = lineas[i].Split(','); // Separa cada línea por comas
12
13     Estudiante estudiante = new Estudiante() // Crea un objeto Estudiante
14     {
15         Id = int.Parse(datos[0]),
16         Nombre = datos[1],
17         Carrera = datos[2]
18     };
19
20     estudiantes.Add(estudiante); // Lo agrega a la lista que ya se había iniciado antes
21 }
22
23 foreach (Estudiante estudiante in estudiantes) // Se muestran los estudiantes en consola
24 {
25     Console.WriteLine($"{estudiante.Id} - {estudiante.Nombre} - {estudiante.Carrera}");
26 }
27
28 JsonSerializerOptions opciones = new JsonSerializerOptions // Aquí se convierte la lista a JSON
29 {
30     WriteIndented = true
31 };
32
33 string json = JsonSerializer.Serialize(estudiantes, opciones);
34
35 File.WriteAllText("estudiantes.json", json); // Por último, se guarda el archivo JSON
36
37 Console.WriteLine("Archivo estudiantes.json creado correctamente.");
```

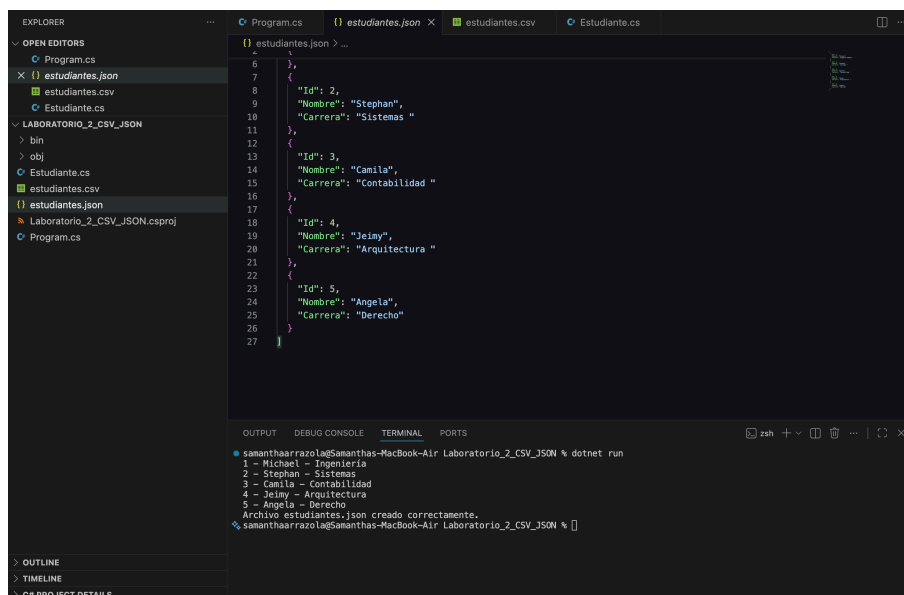
5. PRUEBAS

Ejecute la aplicación y verifique que en la consola aparezca:

- 1 - Ana - Ingeniería
- 2 - Carlos - Sistemas
- 3 - María - Contabilidad
- 4 - Pedro - Arquitectura
- 5 - Sofia - Derecho

Archivo estudiantes.json creado correctamente.

Verifique además que en la carpeta del proyecto exista el archivo: estudiantes.json



```
{
  "Id": 2,
  "Nombre": "Stephan",
  "Carrera": "Sistemas "
},
{
  "Id": 3,
  "Nombre": "Camila",
  "Carrera": "Contabilidad "
},
{
  "Id": 4,
  "Nombre": "Jeimy",
  "Carrera": "Arquitectura "
},
{
  "Id": 5,
  "Nombre": "Angela",
  "Carrera": "Derecho"
}
```

```
1 - Michael - Ingeniería
2 - Stephan - Sistemas
3 - Camila - Contabilidad
4 - Jeimy - Arquitectura
5 - Angela - Derecho
Archivo estudiantes.json creado correctamente.
samanthaarrazola@Samanthas-MacBook-Air Laboratorio_2_CSV_JSON %
```

6. RESPONDER

1. ¿Qué función cumple `File.ReadAllLines()`?

`File.ReadAllLines()` se utiliza para leer todas las líneas de un archivo de texto y almacenarlas en un arreglo de cadenas, es decir, un `string[]`. Cada posición del arreglo corresponde a una línea del archivo y esto facilita recorrer y procesar su contenido. En este lab se utilizó para leer la información del archivo `estudiantes.csv`.

2. ¿Para qué sirve `Split(',')`?

`Split(',')` sirve para dividir una cadena de texto utilizando la coma como separador, como bien está indicado en los comentarios del código. De esta manera, es posible obtener cada dato por separado. En este lab se utilizó para separar el ID, el nombre y la carrera de cada estudiante contenidos en una línea del archivo CSV.

3. ¿Qué ventaja tiene utilizar una `List<Estudiante>`?

La ventaja principal de utilizar una `List<Estudiante>` es que permite almacenar varios objetos del mismo tipo de forma organizada y dinámica, sin la necesidad de definir un tamaño fijo. Asimismo, hace más fácil el proceso de agregar nuevos elementos, recorrer la información y trabajar posteriormente con los datos, como convertir la lista a formato JSON o mostrarla en la consola.

4. ¿Qué es JSON y para qué se utiliza?

Según lo investigado, JSON (JavaScript Object Notation) es un formato de intercambio de datos que permite almacenar información de manera estructurada y fácil de interpretar tanto por personas como por programas. Se usa para intercambiar información entre aplicaciones, desarrollar servicios web (APIs), guardar configuraciones y almacenar datos en un formato ligero y compatible con diferentes lenguajes de programación.

7. ENTREGABLE

Suba a la plataforma: Link repositorio remoto, semana_2 folder remoto debe contener:

- Programa completo en `c#`
- Archivo `estudiantes.csv`
- Archivo `estudiantes.json` generado
- Captura pantalla ejecución de programa mostrando salida en consola